Name:

PRN:

Experiment no 11

Title: CUDA Performance Optimization and cuBLAS

1. AIm

To optimize a CUDA matrix multiplication program and compare its performance with the cuBLAS library, understanding how memory access, thread-block configuration, and optimized libraries influence GPU performance.

2. Why Are We Learning This?

Matrix multiplication is widely used in deep learning, scientific computing, image processing, and numerical simulations. In this assignment, you will compare a custom CUDA implementation with a highly optimized library implementation.

3. Learning Focus

- CUDA thread-block configuration
- Shared-memory optimization
- GPU execution-time measurement
- cuBLAS library usage
- CPU vs GPU performance
- Speedup and GFLOPS calculation

4. Software and Hardware Requirements

- C/C++ compiler
- NVIDIA CUDA Toolkit and nvcc
- NVIDIA CUDA-capable GPU
- cuBLAS library
- Google Colab with GPU enabled or another compatible NVIDIA GPU environment

Record the GPU model, CUDA version, CPU details, and available memory before starting.

Task 1 – Optimize CUDA Matrix Multiplication

Step 1: Sequential CPU implementation

Write a standard C/C++ program to multiply two matrices and measure its execution time.

Step 2: Basic CUDA implementation

Write a CUDA kernel in which each thread calculates one element of the output matrix. Use global memory to access matrix elements.

Step 3: Optimized CUDA implementation

Improve the basic kernel using shared-memory tiling. Load tiles of matrices into shared memory, synchronize threads, and reuse the loaded data during multiplication.

Experiments

Test matrix sizes of:

- 256 × 256
- 512 × 512
- 1024 × 1024

Experiment with block/tile sizes of 8 × 8 and 16 × 16. Try 32 × 32 only if supported by the GPU and implementation.

Record Your Results

Matrix Size	CPU Time (ms)	Basic CUDA (ms)	Tiled CUDA (ms)
256 × 256			
512 × 512			
1024 × 1024			

Measure CUDA kernel time separately from total execution time, including memory transfers.

Analyze

1. Which CUDA implementation performed better?
2. Did shared-memory tiling improve performance for every matrix size?
3. How did the block/tile size affect execution time?
4. Why can a GPU be slower than a CPU for small matrices?

Task 2 – Matrix Multiplication Using cuBLAS

Problem Statement

Implement the same matrix multiplication using the **cuBLAS** library and compare it with your custom CUDA implementation.

Implementation

1. Reuse the matrix inputs from Task 1.
2. Allocate GPU memory for the input and output matrices.
3. Create a cuBLAS handle.
4. Perform matrix multiplication using cublasSgemm() for single-precision floating-point matrices.
5. Copy the result back to the CPU.
6. Verify the result against the CPU reference.
7. Measure the library execution time.

Important: cuBLAS uses column-major matrix conventions by default. Handle the matrix layout and leading dimensions correctly so that the result represents.

Record Your Results

Matrix Size	Basic CUDA (ms)	Tiled CUDA (ms)	cuBLAS (ms)
256 × 256			
512 × 512			
1024 × 1024			

Use consistent timing methodology. Report kernel/library execution time separately from end-to-end time wherever possible.

Performance Metrics

Calculate speedup relative to the sequential CPU implementation:

Calculate approximate floating-point performance

Analyze

1. Did cuBLAS outperform your custom CUDA kernels?
2. Why might cuBLAS achieve better performance?
3. How does matrix size affect the observed speedup?
4. Why should correctness be checked before comparing performance?
5. When would you prefer cuBLAS over writing a custom CUDA kernel?

5. Performance Graphs

Prepare these two graphs:

1. **Matrix size vs execution time** — compare CPU, basic CUDA, tiled CUDA, and cuBLAS.
2. **Matrix size vs speedup** — compare the CUDA implementations and cuBLAS against the CPU baseline.

6. Answer briefly:

1. What was the most effective optimization in Task 1?
2. What is the main difference between a custom CUDA kernel and cuBLAS?
3. Why does GPU performance depend on both computation and memory movement?
4. What did this experiment teach you about selecting an implementation for an HPC application?

7. Deliverables

Submit:

- CPU, basic CUDA, tiled CUDA, and cuBLAS source code.
- Hardware and software configuration.
- Correctness verification results.
- Execution-time and speedup tables.
- Two performance graphs.
- A short conclusion based on your measurements.